



Universitat d'Alacant
Universidad de Alicante

Memoria Práctica de Seguridad y Confidencialidad

Sistema de Salud Seguro con Data Space — Sprout



Universitat d'Alacant
Universidad de Alicante

Desarrollado por:

López Zaragoza, Javier

DNI: 29520088W

Patiño Casas, Jesús

DNI: 06333937J

Mayo 2026



ÍNDICE

1. Introducción	3
2. Manual de usuario	4
2.1 Requisitos y arranque	4
2.2 Flujo de login con 2FA	4
2.3 Menú del Administrador	4
2.4 Menú del Médico / Hospital	4
2.5 Menú del Investigador	5
2.6 Menú del Paciente	5
2.7 Notificaciones automáticas	5
2.8 Funcionalidades por modo de ejecución	6
3. Seguridad	7
3.1 Otros problemas de seguridad abordados	10
4. Funcionalidad	12
4.1 Funcionalidades esperadas en un Data Space	12
4.2 Funcionalidades desarrolladas	12
4.3 Tabla comparativa	13
5. Implementación	14
5.1 Cifrado y compresión de la base de datos	14
5.2 Gestión de contraseñas en la base de datos	14
5.3 Verificación de acceso por rol	14
5.4 Proceso de registro de un nuevo usuario	14
5.5 Securitización del canal	15
5.6 Autenticación en dos factores	15
5.7 Historial de versiones de registros clínicos	15
6. Conclusiones	17



1. Introducción

Esta práctica parte de Sprout, un scaffold básico en Go que implementa un servidor HTTP con API JSON mínima, un cliente de terminal y persistencia clave/valor basada en bbolt. El código base presentaba múltiples carencias de seguridad deliberadas: contraseñas en texto plano, tokens predecibles, comunicación HTTP sin cifrar y base de datos sin protección.

El trabajo realizado transforma ese prototipo en un sistema de gestión de datos clínicos que permite a hospitales administrar pacientes de forma local y contribuir datos de forma anónima a un repositorio de investigación (Data Space). A lo largo del proceso se han abordado los problemas de seguridad fundamentales del enunciado, se ha diseñado una arquitectura de doble base de datos que garantiza la separación entre datos identificados y datos anónimos, y se han implementado mejoras adicionales como autenticación en dos factores, bloqueo por intentos fallidos, rate limiting, firma de integridad de datos, historial de versiones y sistema de notificaciones por rol.

Los conceptos aprendidos —cifrado simétrico, hash de contraseñas, JWT, TLS, HMAC, anonimización de datos— tienen aplicación directa en cualquier sistema que maneje información sensible. La arquitectura desarrollada, con separación física de bases de datos y comunicación unidireccional entre nodos, reproduce patrones reales de los espacios de datos sanitarios europeos como Gaia-X Health o el European Health Data Space.



2. Manual de usuario

2.1 Requisitos y arranque

La aplicación requiere Go instalado en el sistema. Desde la raíz del proyecto:

```
mkdir data          # Crear carpeta de datos (solo la primera vez)
go run main.go      # Arrancar la aplicación
```

Al arrancar, el sistema pide seleccionar el modo de ejecución:

- **Servidor central** — levanta únicamente el servidor HTTPS en :8080, sin cliente interactivo. Gestiona el Data Space global y los usuarios investigadores/admin.
- **Hospital / Cliente** — levanta el servidor local con su propia base de datos y el cliente interactivo de terminal. Permite elegir entre servidor local o remoto.

A continuación, se pide la contraseña maestra de la base de datos. Si la base de datos no existe, se crea con el usuario administrador por defecto (usuario: admin, contraseña: admin). En el primer login se recomendará cambiarla.

2.2 Flujo de login con 2FA

El proceso de autenticación tiene dos fases. Primero se introducen usuario y contraseña. Si son correctos, el servidor imprime un código de seis dígitos en su consola (simulando el envío por SMS). El cliente pide ese código para completar el login. Solo tras introducirlo correctamente se emite el token de sesión.

2.3 Menú del Administrador

El administrador dispone de las siguientes opciones:

- Listar usuarios — muestra todos los usuarios con su rol, estado (bloqueado, sin consentimiento, solicita eliminación) y nombre de hospital si procede.
- Dar de alta médico/hospital, investigador o paciente — crea usuarios con contraseña inicial y rol asignado.
- Eliminar usuario — elimina cuenta y datos asociados.
- Cambiar rol de usuario — modifica el rol de cualquier usuario existente.
- Ver consultas de investigadores — lista todas las peticiones con sus filtros y estado.
- Autorizar petición — aprueba o deniega consultas de investigadores o solicitudes de eliminación de pacientes.
- Desbloquear cuenta — reactiva cuentas bloqueadas por exceso de intentos fallidos.
- Cambiar contraseña.

2.4 Menú del Médico / Hospital



- Introducir datos de paciente — formulario guiado que genera XML clínico validado y lo guarda en la base de datos local del hospital con identificadores completos.
- Ver mis pacientes — lista todos los registros del hospital.
- Buscar paciente concreto — recupera el registro de un paciente por su ID.
- Ver historial de versiones — muestra las versiones anteriores del registro de un paciente antes de cada actualización (máximo 10 versiones).
- Subir registros al servidor central — anonimiza los datos (elimina identificadores, añade IDHash y firma HMAC) y los envía al servidor central. Puede subir todos o uno concreto.
- Configurar URL del servidor central — permite cambiar la dirección del servidor central sin recompilar.

2.5 Menú del Investigador

- Hacer petición de consulta — describe la consulta y los filtros deseados (diagnóstico, sexo, rango de edad). El administrador debe aprobarla.
- Ver mis consultas — lista las peticiones filtradas por estado (pendientes, aprobadas, denegadas).
- Ejecutar consulta aprobada — accede al Data Space con los filtros aprobados y obtiene estadísticas y registros anónimos.
- Exportar resultados — guarda los resultados en un fichero local con marca de tiempo.

2.6 Menú del Paciente

- Ver mis datos clínicos — muestra el registro XML almacenado.
- Revocar permisos de uso de datos — impide que el médico envíe sus datos al Data Space.
- Solicitar eliminación de datos — envía al administrador una petición de borrado de la cuenta.
- Cambiar contraseña.

2.7 Notificaciones automáticas

Al inicio de cada ciclo del menú, el sistema muestra automáticamente avisos relevantes para el rol del usuario: cuentas bloqueadas y consultas pendientes para el administrador, consultas aprobadas listas para ejecutar para el investigador, y número de pacientes en base local para el médico.



2.8 Funcionalidades por modo de ejecución

El sistema se ejecuta en dos modos distintos que comparten el mismo binario, pero utilizan bases de datos separadas y exponen conjuntos de funcionalidades distintos. Esta separación no es solo organizativa: garantiza por construcción que el servidor central nunca tenga acceso a datos identificados y que el hospital nunca pueda leer del repositorio anónimo.

Modo hospital arranca en el puerto 8080 y utiliza data/hospital.db. Contiene la lógica de gestión local de pacientes: el médico puede guardar registros clínicos completos con identificadores, consultarlos, buscar pacientes concretos y revisar el historial de versiones anteriores de cada registro. Cuando el médico decide contribuir datos al repositorio de investigación, el sistema anonimiza el registro en este mismo nodo antes de enviarlo, de forma que el servidor central nunca recibe ningún identificador. El paciente también opera contra este nodo: puede consultar sus datos, revocar el consentimiento y solicitar la eliminación de su cuenta.

Modo servidor central arranca en el puerto por defecto, después se puede cambiar y utiliza data/server.db. Solo acepta datos ya anonimizados procedentes de los hospitales y expone el repositorio de investigación a los investigadores autorizados. Desde este nodo el investigador puede solicitar consultas con filtros, el administrador puede aprobarlas o denegarlas, y el investigador puede ejecutar las aprobadas y exportar los resultados. No existe ningún endpoint que permita recuperar datos por hospital o por paciente, porque ese concepto no existe en el modelo de datos del servidor central.

Acciones comunes a ambos modos: registro, login con 2FA, logout, cambio de contraseña, renovación de token y todas las operaciones de administración de usuarios (dar de alta, eliminar, cambiar rol, desbloquear cuentas, listar usuarios). Esto permite que cada nodo tenga su propio administrador independiente.

Login, registro, logout, cambio de contraseña y gestión de usuarios (crear, eliminar, cambiar rol, desbloquear) están disponibles en ambos modos.

En el hospital están disponibles: guardar registros de pacientes en la base de datos local, consultar y buscar pacientes propios, ver el historial de versiones de un registro, revocar el consentimiento por parte del paciente, solicitar la eliminación de cuenta y enviar datos anonimizados al servidor central.

En el servidor central están disponibles: recibir datos anónimos de los hospitales, consultar el Data Space, solicitar y gestionar consultas de investigación y exportar los resultados.

3. Seguridad

A continuación, se describen los problemas de seguridad abordados, siguiendo el esquema requerido en la plantilla.

Confidencialidad de la base de datos

El sistema utiliza bbolt (ficheros store.go y bbolt.go) para persistir toda la información del sistema. En el código base, los datos se almacenaban en texto plano.

Problemática

Al guardar los datos sin cifrar, cualquier persona con acceso físico al fichero .db puede leerlos directamente con un editor hexadecimal o herramientas específicas de bbolt, accediendo a contraseñas, datos clínicos y sesiones activas.

Solución propuesta

Se modificó bbolt.go para cifrar cada valor antes de escribirlo en disco y descifrarlo al leerlo. El cifrado elegido es AES-256 en modo CTR con clave derivada de SHA-256 sobre la contraseña maestra. Adicionalmente se aplica compresión zlib antes del cifrado para reducir el tamaño. La clave maestra se pide al arrancar el servidor y no se almacena en ningún sitio.

```
func (s *BboltStore) protect(data []byte) ([]byte, error) {
    h := sha256.Sum256(s.key)           // clave AES-256
    block, _ := aes.NewCipher(h[:])
    iv := sha256.Sum256([]byte("<inicializar>"))
    stream := cipher.NewCTR(block, iv[:16])
    var out bytes.Buffer
    sw := &cipher.StreamWriter{S: stream, W: &out}
    zw := zlib.NewWriter(sw)           // compresión
    zw.Write(data)
    zw.Close()
    return out.Bytes(), nil
}
```

Confidencialidad del almacenamiento de contraseñas

En el código base, las contraseñas se guardaban en texto plano en la base de datos, lo que las exponía directamente en caso de compromiso del fichero.

Problemática

Si alguien accede a la base de datos, puede leer todas las contraseñas sin ningún esfuerzo adicional. Aunque la base de datos esté cifrada, el administrador del sistema podría ver las contraseñas en texto plano durante la operación normal.

Solución propuesta

Se implementó hash de contraseñas usando bcrypt con sal aleatoria de 16 bytes por usuario. bcrypt está diseñado para ser computacionalmente costoso, haciendo inviable la fuerza bruta y los ataques de diccionario incluso con hardware especializado. La sal impide el uso de tablas arcoíris.

```
func (s *server) hashPassword(pw string) ([]byte, []byte, error) {
    salt := make([]byte, 16)
    rand.Read(salt) // sal criptográficamente segura
    hash, err := bcrypt.Key([]byte(pw), salt, 16384, 8, 1, 32)
    return hash, salt, err
}
```



Autenticación y control de acceso por roles

El código base no diferenciaba usuarios ni roles. Cualquier usuario autenticado podía ejecutar cualquier operación del servidor.

Problemática

Sin control de acceso por roles, un usuario normal podría realizar operaciones de administración, acceder a datos de otros usuarios o consultar el Data Space sin autorización.

Solución propuesta

Se implementó un sistema de cuatro roles (admin, médico, investigador, paciente) con menús y acciones distintas. El rol queda firmado dentro del token JWT. El servidor verifica el rol en cada petición sin necesidad de consultar la base de datos.

```
func (s *server) requireRole(tok string, roles ...string) (string, string, bool)
{
    username, role, _, valid := s.validateJWT(tok)
    if !valid { return "", "", false }
    for _, r := range roles {
        if role == r { return username, role, true }
    }
    return "", "", false
}

// Uso en cada handler:
caller, _, ok := s.requireRole(req.Token, api.RoleAdmin)
if !ok { return api.Response{Success: false, Message: "Acceso denegado"} }
```

Sobreexposición de clave de login — uso de token JWT

El código base usaba un contador atómico para generar tokens del tipo token_1, token_2, etc. Estos tokens son predecibles y no tienen caducidad.

Problemática

Un atacante puede probar tokens de forma secuencial hasta encontrar uno válido. Al no tener expiración, un token robado sirve indefinidamente.

Solución propuesta

Se sustituyeron los contadores por tokens JWT firmados con HMAC-SHA256. Cada token incluye usuario, rol y fecha de expiración (1 hora). El cliente renueva el token automáticamente cuando quedan menos de 10 minutos. Adicionalmente, se invalida la sesión si hay más de 30 minutos de inactividad aunque el JWT siga vigente.

```
func (s *server) generateJWT(username, role string) (string, error) {
    claims := &Payload{
        Username: username, Role: role,
        StandardClaims: jwt.StandardClaims{
            ExpiresAt: time.Now().Add(1 * time.Hour).Unix(),
        },
    }
    return jwt.NewWithClaims(jwt.SigningMethodHS256, claims).
        SignedString([]byte(jwtSecret))
}
```

Gestión de la clave maestra de cifrado

La clave de cifrado de la base de datos debe derivarse de algún secreto. Almacenarla en código fuente o en un fichero de configuración la expone a cualquiera que tenga acceso al repositorio.



Problemática

Si la clave está en el código o en un fichero, cualquier persona que obtenga el repositorio o el binario puede descifrar la base de datos.

Solución propuesta

La clave maestra se pide al usuario en cada arranque del servidor mediante prompt seguro (sin eco en pantalla). Se aplica SHA-256 para derivar una clave AES de 256 bits. El servidor genera una marca de validación cifrada en la primera ejecución; si la clave maestra es incorrecta en arranques posteriores, el servidor no arranca.

```
// main.go
pass, _ := ui.ReadPassword("Contraseña maestra de la base de datos")
h := sha256.Sum256([]byte(pass))
masterKey := h[:]

// server.go - verificación al arrancar
val, err := db.Get("system", []byte("master_check"))
if err != nil || string(val) != "VALIDO_2026" {
    db.Close()
    return fmt.Errorf("CLAVE MAESTRA INCORRECTA")
}
```

Securización del canal de comunicación — HTTPS con TLS

El código base usaba `http.ListenAndServe()`, transmitiendo toda la información en texto plano sobre la red.

Problemática

Cualquier persona en la misma red puede capturar el tráfico con herramientas como Wireshark y leer credenciales, tokens y datos clínicos sin ningún esfuerzo.

Solución propuesta

Se sustituyó `ListenAndServe` por `ListenAndServeTLS` con certificado autofirmado (`cert.pem` y `key.pem`). El cliente configura TLS con `InsecureSkipVerify` para aceptar el certificado autofirmado durante el desarrollo. En producción se usaría un certificado emitido por una CA reconocida.

```
// server.go
return http.ListenAndServeTLS(":8080", "cert.pem", "key.pem", mux)

// client.go
Transport: &http.Transport{
    TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
}
```



3.1 Otros problemas de seguridad abordados

Autenticación en dos factores (2FA)

Una contraseña robada permite el acceso completo al sistema sin ninguna barrera adicional.

Problemática

Si la contraseña de un usuario queda expuesta por phishing, ingeniería social o reutilización de credenciales, el atacante tiene acceso inmediato a todos los datos.

Solución propuesta

El login se divide en dos fases. Tras verificar la contraseña, el servidor genera un código de 6 dígitos con crypto/rand y lo muestra en su log (simulando envío por SMS). Emite un token temporal de 5 minutos. El cliente lo pide al usuario y lo envía en una segunda petición. Solo si coincide se emite el JWT definitivo.

Bloqueo de cuenta por intentos fallidos

Sin límite de intentos, un atacante puede probar contraseñas ilimitadamente de forma automatizada (ataque de fuerza bruta online).

Problemática

Un script puede probar millones de contraseñas contra el endpoint de login sin ninguna fricción, especialmente si las contraseñas son débiles.

Solución propuesta

Tras 5 intentos fallidos consecutivos, la cuenta queda bloqueada durante 15 minutos. El contador persiste en la base de datos y sobrevive a reinicios del servidor. El administrador puede desbloquear cuentas manualmente. El sistema de notificaciones alerta al administrador cuando hay cuentas bloqueadas.

Rate limiting por dirección IP

Incluso con bloqueo de cuenta, un atacante puede intentar comprometer múltiples cuentas distintas o explorar la API de forma automatizada.

Problemática

Un atacante puede distribuir los intentos entre muchas cuentas para evitar el bloqueo individual, o simplemente explorar endpoints para detectar vulnerabilidades.

Solución propuesta

El servidor mantiene un mapa de contadores por IP protegido por mutex. Cada IP tiene un máximo de 20 peticiones por minuto. Al superarlo, el servidor devuelve HTTP 429 sin ejecutar ninguna lógica de negocio, registrando el evento en el log de seguridad.

Firma HMAC-SHA256 de datos clínicos en tránsito

TLS protege el canal pero no verifica que el contenido no haya sido manipulado en el punto de origen o por un intermediario malicioso antes del cifrado.

Problemática

Un proxy o middleware malicioso podría modificar los datos clínicos antes de que lleguen al servidor central, introduciendo información falsa en el repositorio de investigación.

Solución propuesta

Antes de subir datos al servidor central, el hospital calcula un HMAC-SHA256 del XML usando un secreto compartido e incluye la firma en el campo DataSig. El servidor



verifica la firma antes de aceptar el registro. Si no coincide, rechaza el dato con error de integridad.

Log de auditoría de seguridad

Sin registro de eventos, es imposible detectar intentos de intrusión, auditar accesos o investigar incidentes de seguridad a posteriori.

Problemática

La ausencia de logs permite que ataques fallidos, accesos no autorizados o comportamientos anómalos pasen desapercibidos indefinidamente.

Solución propuesta

Todos los eventos de seguridad relevantes se escriben en data/security.log con marca de tiempo: logins exitosos y fallidos, bloqueos y desbloqueos de cuenta, verificaciones 2FA, rate limiting, cambios de contraseña, renovaciones de token, expiración por inactividad, subidas al Data Space y consultas de investigadores.

Cambio obligatorio de contraseña periódico

Las contraseñas iniciales asignadas por el administrador suelen ser débiles o compartidas, y los usuarios tienden a no cambiarlas por iniciativa propia.

Problemática

El uso indefinido de contraseñas débiles o comprometidas aumenta el riesgo de acceso no autorizado con el tiempo.

Solución propuesta

Al crear una cuenta, la contraseña se marca como provisional. El sistema lleva un contador de logins y tras cada 3 accesos notifica al usuario que debe cambiar su contraseña desde la opción del menú.

4. Funcionalidad

4.1 Funcionalidades esperadas en un Data Space

Un Data Space sanitario real (como el European Health Data Space o Gaia-X Health) incorpora las siguientes características fundamentales:

- Control de acceso federado con múltiples roles (paciente, proveedor de datos, consumidor, árbitro).
- Portabilidad de datos en formatos estandarizados (HL7 FHIR, ICD-10, SNOMED-CT).
- Anonimización y seudonimización de datos antes de compartirlos.
- Consentimiento granular del titular de los datos.
- Trazabilidad completa de accesos y modificaciones.
- Nodos federados que comparten datos sin centralizar la información.
- Mecanismos de deduplicación para evitar registros redundantes.
- Protección de anonimato mediante umbrales mínimos de agregación.
- Auditoría continua y registro de eventos.
- Cifrado en tránsito y en reposo.

4.2 Funcionalidades desarrolladas

Sprout implementa las siguientes funcionalidades alineadas con el modelo de Data Space:

- Arquitectura de doble base de datos: hospital (datos identificados) y servidor central (datos anónimos), con comunicación unidireccional.
- Sistema de cuatro roles con menús y permisos distintos: administrador, médico/hospital, investigador y paciente.
- Formato XML clínico estructurado con validación de campos (edad, sexo, diagnóstico).
- Consentimiento revocable por el paciente que bloquea el envío de sus datos al Data Space.
- Anonimización por identificador aleatorio (32 bytes de crypto/rand) al subir al servidor central.
- Deduplicación por IDHash (SHA-256 del ID del paciente) para evitar duplicados en el repositorio.
- Filtros de consulta por diagnóstico, sexo y rango de edad, con protección por umbral mínimo de registros.
- Historial de versiones de registros clínicos (hasta 10 versiones por paciente).
- Cifrado AES-256 en reposo, HTTPS en tránsito y firma HMAC de datos clínicos.
- Log de auditoría completo de eventos de seguridad.
- Sistema de notificaciones por rol al inicio de cada sesión.

4.3 Tabla comparativa

Data Space (referencia)	Sprout (implementado)	Observaciones
Control de acceso federado por roles	Sistema de 4 roles (admin, médico, investigador, paciente) con JWT firmado	Los roles quedan firmados en el token; el servidor los verifica sin consultar la BD en cada petición
Portabilidad en formatos estándar (HL7, FHIR)	XML clínico propio con campos validados	No se usa FHIR ni ICD-10 pero la estructura XML es extensible a esos estándares
Anonimización antes de compartir	ID aleatorio de 32 bytes + eliminación de identificadores + IDHash	La anonimización ocurre en el hospital antes de enviar; el servidor central no recibe ningún identificador
Consentimiento del titular de los datos	Revocación de consentimiento por el paciente	El paciente puede revocar el uso de sus datos e impedir envíos futuros al Data Space
Nodos federados	Modo hospital + modo servidor central como nodos distintos	Cada hospital tiene su propia BD local; el servidor central actúa como nodo agregador
Deduplicación de registros	IDHash (SHA-256 del ID del paciente)	El hash permite detectar duplicados sin revelar el ID original
Protección de anonimato por umbral	Umbral mínimo configurable (defecto: 5 registros)	Si la consulta devuelve menos de N registros, se rechaza para evitar reidentificación
Trazabilidad y auditoría	Log de auditoría en data/security.log	Cubre logins, cambios, subidas, consultas e incidentes de seguridad
Cifrado en reposo	AES-256-CTR + zlib en bbolt.go	Cada valor se cifra individualmente antes de escribirse en disco
Cifrado en tránsito	HTTPS con TLS + HMAC de datos clínicos	TLS protege el canal; HMAC garantiza integridad del contenido
Gestión de solicitudes de eliminación (derecho al olvido)	Solicitud de borrado por paciente, aprobación por admin	El flujo respeta el principio de que el borrado requiere supervisión

5. Implementación

5.1 Cifrado y compresión de la base de datos

Las funciones `protect` y `unprotect` en `pkg/store/bbolt.go` interceptan todas las escrituras y lecturas de la base de datos. Al guardar, se comprime primero con `zlib` y luego se cifra con `AES-256-CTR`. Al leer, se descifra y descomprime en orden inverso.

```
// Escribir: primero comprimir, luego cifrar
zw := zlib.NewWriter(sw) // sw es un cipher.StreamWriter con AES-CTR
zw.Write(data)
zw.Close()

// Leer: primero descifrar, luego descomprimir
sr := &cipher.StreamReader{S: stream, R: bytes.NewReader(data)}
zr, _ := zlib.NewReader(sr)
return io.ReadAll(zr)
```

5.2 Gestión de contraseñas en la base de datos

Las contraseñas se procesan en `server.go`. Nunca se almacena la contraseña en texto plano. La función `hashPassword` genera una sal aleatoria y aplica `scrypt`. En el login, se recalcula el hash con la sal almacenada y se compara con `bytes.Equal` para evitar ataques de tiempo.

```
// Registro: generar hash + sal
hash, salt, _ := s.hashPassword(req.Password)
entry := authEntry{Hash: hash, Salt: salt, Role: role, ...}

// Login: verificar sin revelar el hash real
h, _ := scrypt.Key([]byte(req.Password), entry.Salt, 16384, 8, 1, 32)
if !bytes.Equal(entry.Hash, h) {
    // credenciales incorrectas
}
```

5.3 Verificación de acceso por rol

Cada endpoint del servidor llama a `requireRole` pasando el token JWT y los roles permitidos. La función valida la firma del token, extrae el rol firmado y comprueba si está en la lista permitida.

```
// Cualquier handler protegido:
caller, _, ok := s.requireRole(req.Token, api.RoleAdmin)
if !ok {
    return api.Response{Success: false, Message: "Acceso denegado"}
}
```

5.4 Proceso de registro de un nuevo usuario

El administrador usa `adminCreateUser` o cualquier usuario puede registrarse con `registerUser`. En ambos casos se valida que el nombre de usuario no exista, se hashea la



contraseña y se marca como provisional (`MustChangePass: true`). El hospital puede especificar su nombre visible al crear un usuario médico.

```
entry := authEntry{
    Hash: hash, Salt: salt,
    Role: role,
    MustChangePass: true,          // provisional, debe cambiarla
    HospitalName: req.HospitalName, // solo para rol médico
}
s.saveAuth(req.Username, entry)
```

5.5 Securización del canal

El servidor arranca con `http.ListenAndServeTLS` usando los ficheros `cert.pem` y `key.pem`. El cliente configura el transporte HTTP con TLS. Los certificados autofirmados se aceptan con `InsecureSkipVerify: true` durante el desarrollo.

```
// server.go - arranque con TLS
return http.ListenAndServeTLS(":8080", "cert.pem", "key.pem", mux)

// client.go - cliente con TLS
httpClient: &http.Client{
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    },
}
```

5.6 Autenticación en dos factores

El servidor mantiene un mapa `pending2FA` en memoria. Al verificar la contraseña, genera un código de 6 dígitos con `crypto/rand` y un token temporal. El cliente lo recibe y pide el código al usuario. Si coincide y no ha expirado, se emite el JWT definitivo y se elimina la entrada del mapa.

```
// Primera fase: verificar contraseña
code := generate2FACode() // 6 dígitos con crypto/rand
tempToken := generateTempToken()
s.pending2FA[tempToken] = pending2FA{code: code,
    expiresAt: time.Now().Add(5 * time.Minute)}
s.secuLog.Printf("2FA_CODE usuario='%s' codigo='%s'", ...)

// Segunda fase: verificar código
p := s.pending2FA[req.TempToken]
if req.Code2FA != p.code { /* incorrecto */ }
// Código correcto: emitir JWT real
```

5.7 Historial de versiones de registros clínicos

En `medicoSaveLocal`, antes de sobrescribir el registro actual, se guarda la versión anterior en el bucket `history` con su timestamp y autor. Se mantienen las últimas 10 versiones.



```
// Guardar versión anterior antes de sobrescribir
existing, err := s.db.Get("patients", []byte(key))
if err == nil {
    histKey := fmt.Sprintf("history|%s|%s", hospital, req.PatientID)
    var versions []patientRecord
    json.Unmarshal(raw, &versions) // cargar historial previo
    versions = append(versions, patientRecord{
        Data: string(existing), Timestamp: time.Now(), Author: hospital,
    })
    if len(versions) > 10 { versions = versions[len(versions)-10:] }
    s.db.Put("history", []byte(histKey), histData)
}
```



6. Conclusiones

La práctica ha permitido afrontar de forma práctica los principales problemas de seguridad de una aplicación real que maneja datos sensibles. Partir de un código base deliberadamente inseguro y tener que identificar y resolver cada vulnerabilidad ha sido mucho más efectivo que estudiar los conceptos de forma teórica.

La dificultad más importante no fue técnica sino de diseño: decidir la arquitectura de doble base de datos para garantizar la separación entre datos identificados y datos anónimos. Una vez tomada esa decisión, las implicaciones se propagaron por todo el sistema: la comunicación unidireccional, el flujo de anonimización en el hospital antes de enviar, la deduplicación por hash irreversible. Entender que la privacidad es una propiedad arquitectural y no solo una capa añadida ha sido el aprendizaje más relevante.

A nivel técnico, los conocimientos más valiosos han sido el funcionamiento real de la derivación de claves con scrypt (especialmente la diferencia entre hash y KDF), la firma HMAC como garantía de integridad independiente del canal, y la gestión del ciclo de vida de tokens JWT con renovación automática e invalidación por inactividad.

Como líneas de trabajo futuras, las más interesantes serían la integración con estándares reales como HL7 FHIR o ICD-10 para los diagnósticos, la implementación del envío del código 2FA por SMS o correo electrónico real, y el desarrollo de un mecanismo de recuperación de contraseña sin intervención del administrador. También sería relevante añadir importación de datos desde CSV para facilitar la carga masiva de registros desde sistemas hospitalarios existentes.